```
----------Main Entity----------
-- Company: Digilent Inc 2011
-- Engineer: Michelle Yu
-- Create Date:    17:05:39 08/23/2011
--
-- Module Name:    PmodKYPD - Behavioral
-- Project Name:  PmodKYPD
-- Target Devices: Nexys3
-- Tool versions: Xilinx ISE 13.2
-- Description:
--          This file defines a project that outputs the key pressed on the PmodKYPD to the seven segment display
--
-- Revision:
-- Revision 0.01 - File Created
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;


entity PmodKYPD is
   Port (
                              clk : in  STD_LOGIC;
                              JA : inout  STD_LOGIC_VECTOR (7 downto 0); -- PmodKYPD is designed to be connected to JA
        an : out  STD_LOGIC_VECTOR (3 downto 0);   -- Controls which position of the seven segment display to display
        seg : out  STD_LOGIC_VECTOR (6 downto 0)); -- digit to display on the seven segment display
end PmodKYPD;


architecture Behavioral of PmodKYPD is
```

```
component Decoder is
        Port (
                        clk : in  STD_LOGIC;
      Row : in  STD_LOGIC_VECTOR (3 downto 0);
                        Col : out  STD_LOGIC_VECTOR (3 downto 0);
      DecodeOut : out  STD_LOGIC_VECTOR (3 downto 0));
        end component;

component DisplayController is
        Port (
                         DispVal : in  STD_LOGIC_VECTOR (3 downto 0);
       anode: out std_logic_vector(3 downto 0);
       segOut : out  STD_LOGIC_VECTOR (6 downto 0));
        end component;

signal Decode: STD_LOGIC_VECTOR (3 downto 0);
begin


        C0: Decoder port map (clk=>clk, Row =>JA(7 downto 4), Col=>JA(3 downto 0),
DecodeOut=> Decode);
        C1: DisplayController port map (DispVal=>Decode, anode=>an, segOut=>seg );


end Behavioral;
```

Decoder Entity

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity Decoder is
   Port (
                              clk : in  STD_LOGIC;
        Row : in  STD_LOGIC_VECTOR (3 downto 0);
                              Col : out  STD_LOGIC_VECTOR (3 downto 0);
        DecodeOut : out  STD_LOGIC_VECTOR (3 downto 0));
end Decoder;

architecture Behavioral of Decoder is

signal sclk :STD_LOGIC_VECTOR(19 downto 0);
begin
            process(clk)
                        begin
                        if clk'event and clk = '1' then
                                    -- 1ms
                                    if sclk = "00011000011010100000" then
                                                --C1
                                                Col<= "0111";
                                                sclk <= sclk+1;
                                    -- check row pins
                                    elsif sclk = "00011000011010101000" then
                                                --R1
                                                if Row = "0111" then
                                                            DecodeOut <= "0001";   --1
                                                --R2
```

```
		elsif Row = "1011" then
			DecodeOut <= "0100"; --4
		--R3
		elsif Row = "1101" then
			DecodeOut <= "0111"; --7
		--R4
		elsif Row = "1110" then
			DecodeOut <= "0000"; --0
		end if;
		sclk <= sclk+1;
-- 2ms
elsif sclk = "00110000110101000000" then
		--C2
		Col<= "1011";
		sclk <= sclk+1;
-- check row pins
elsif sclk = "00110000110101001000" then
		--R1
		if Row = "0111" then
			DecodeOut <= "0010"; --2
		--R2
		elsif Row = "1011" then
			DecodeOut <= "0101"; --5
		--R3
		elsif Row = "1101" then
			DecodeOut <= "1000"; --8
		--R4
		elsif Row = "1110" then
			DecodeOut <= "1111"; --F
		end if;
		sclk <= sclk+1;
```

```
--3ms
elsif sclk = "01001001001111100000" then
		--C3
		Col<= "1101";
		sclk <= sclk+1;
-- check row pins
elsif sclk = "01001001001111101000" then
		--R1
		if Row = "0111" then
			DecodeOut <= "0011"; --3
		--R2
		elsif Row = "1011" then
			DecodeOut <= "0110"; --6
		--R3
		elsif Row = "1101" then
			DecodeOut <= "1001"; --9
		--R4
		elsif Row = "1110" then
			DecodeOut <= "1110"; --E
		end if;
		sclk <= sclk+1;
--4ms
elsif sclk = "01100001101010000000" then
		--C4
		Col<= "1110";
		sclk <= sclk+1;
-- check row pins
elsif sclk = "01100001101010001000" then
		--R1
		if Row = "0111" then
			DecodeOut <= "1010"; --A
```

```
                --R2
                elsif Row = "1011" then
                        DecodeOut <= "1011"; --B
                --R3
                elsif Row = "1101" then
                        DecodeOut <= "1100"; --C
                --R4
                elsif Row = "1110" then
                        DecodeOut <= "1101"; --D
                end if;
                sclk <= "00000000000000000000";
            else
                sclk <= sclk+1;
            end if;
        end if;
    end process;

end Behavioral;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;


entity DisplayController is
   Port (
                --output from the Decoder
                DispVal : in  STD_LOGIC_VECTOR (3 downto 0);
                --controls the display digits
                anode: out std_logic_vector(3 downto 0);
```

```
                                --controls which digit to display
        segOut : out  STD_LOGIC_VECTOR (6 downto 0));
end DisplayController;
```

DisplayController Entity

```
architecture Behavioral of DisplayController is
begin
        -- only display the leftmost digit
        anode<="1110";


         with DispVal select
                segOut <=  "1000000" when "0000", --0
                                        "1111001" when "0001", --1
                                        "0100100" when "0010", --2
                                        "0110000" when "0011", --3
                                        "0011001" when "0100", --4
                                        "0010010" when "0101", --5
                                        "0000010" when "0110", --6
                                        "1111000" when "0111", --7
                                        "0000000" when "1000", --8
                                        "0010000" when "1001", --9
                                        "0001000" when "1010", --A
                                        "0000011" when "1011", --B
                                        "1000110" when "1100", --C
                                        "0100001" when "1101", --D
                                        "0000110" when "1110", --E
                                        "0001110" when "1111", --F
                                        "0111111" when others;

end Behavioral;
```